



Las siete herramientas del agente

Guía 4 · Taller 02 de caso aplicado de IA en industria

Los ojos del agente. Viven en `agente/herramientas.py` y son lo único que el modelo puede mirar del mundo.

La lista

HERRAMIENTA	QUÉ DEVUELVE	CUÁNDO LA LLAMA
<code>listar_modelos</code>	El inventario: cuántos, de qué categoría y región	Primero, para saber sobre qué trabaja
<code>resumen_flota</code>	Salud de los 192: error, sesgo, cobertura y los peores	El punto de partida de cualquier diagnóstico
<code>agregado_por</code>	Las métricas promediadas por categoría, tienda o región	Para ver la forma del problema
<code>comparar_periodos</code>	Una ventana reciente contra una línea base, con test estadístico	Para decidir si algo cambió de verdad
<code>detectar_anomalias</code>	Series cortadas, tiendas mudas, huecos en los datos	Antes de culpar al modelo
<code>estado_del_job</code>	Si el job corrió, cuánto tardó, cuántas filas entregó	Un modelo puede estar sano y el job caído
<code>detalle_modelo</code>	Las métricas diarias de un modelo concreto	Cuando ya sabes cuál mirar de cerca

Las tres reglas de una herramienta de percepción

1. **Devuelve agregados, no filas.** Decenas de números, no miles. Un modelo de lenguaje no puede leer 17,472 filas — y aunque pudiera, no debe.
2. **Es de solo lectura.** En producción nadie le da escritura a un agente sobre los modelos.
3. **La estadística la hace Python, no el LLM.** El modelo razona sobre los números; no los calcula.

Y una cuarta, que se descubre usándolas: la **docstring es prompt**. No es documentación. Es lo único que el modelo lee para decidir cuándo llamar cada herramienta, así que se escribe como se escribe un prompt.

agregado_por revela la forma

Es la más útil para diagnosticar, y la razón es que la misma degradación se ve distinta según por dónde la cortes:

- Se concentra en una **categoría** y aparece en todas las tiendas → la causa es del producto.
- Se concentra en una **tienda** y toca todas las categorías → la causa es de esa tienda.
- Un modelo suelto degradado → es otra cosa.

Y hay una trampa que vas a ver el sábado. Con la flota **sana**, agrupando por región:

ORIENTE	8 modelos	mape 14.6	sesgo +8.7%
LIMA	112 modelos	mape 13.8	sesgo -0.3%
CENTRO	8 modelos	mape 10.9	sesgo +2.1%

Los dos grupos de ocho modelos ocupan los dos bordes de la tabla, y nada está roto. **El ruido no tiene signo: tiene tamaño de muestra**. Por eso un umbral único marca como rotos a los grupos chicos todos los días.

Una herramienta que falla bien

```
→ detalle_modelo("panaderia_callao")

{
  "error": "No existe el modelo 'panaderia_callao'.",
  "formato": "dem-<categoria>-<tienda>, por ejemplo dem-panaderia-callao",
  "quiza_buscabas": ["dem-panaderia-callao"]
}
```

El modelo se inventó el identificador —guion bajo donde iba guion— y eso va a pasar. Lo que importa no es que falle: es qué recibe cuando falla. Un «no existe» a secas invita a reintentar con otra variante del nombre, y ahí se van cinco llamadas.

Una herramienta que falla muda convierte una llamada equivocada en tres.

Probarlas sin agente

No hace falta un LLM para mirar lo que devuelven. La API está detrás:

```
curl -s "http://localhost:8000/v1/metricas?categoria=bebidas" | head
curl -s "http://localhost:8000/v1/modelos" | head
```

Y en `http://localhost:8000/docs` está la misma API con la interfaz de Swagger, que para explorar es más cómoda.